Parallel and Distributed Computing

Assignment 7 — CUDA Part II

Submitted To	Dr. Saif Nalband
Submitted By	Nitin Gupta
Roll No.	102303525
Platform	Google Colab (PyCUDA)
GPU Used	NVIDIA Tesla T4

PROBLEM 1

Different thread tasks for sum of first n integers

Ques:

Write a CUDA program where all threads perform different tasks for the summation of first n integers ($n = 1024$). One thread uses the iterative approach and another uses the direct formula. Follow the standard steps for host/device memory management and kernel launch.

Output

A single kernel is launched with exactly 2 threads. Thread 0 computes the sum iteratively by looping over all N values; Thread 1 uses the closed-form formula $N(N+1)/2$. Both results are written to separate output slots and verified on the host.

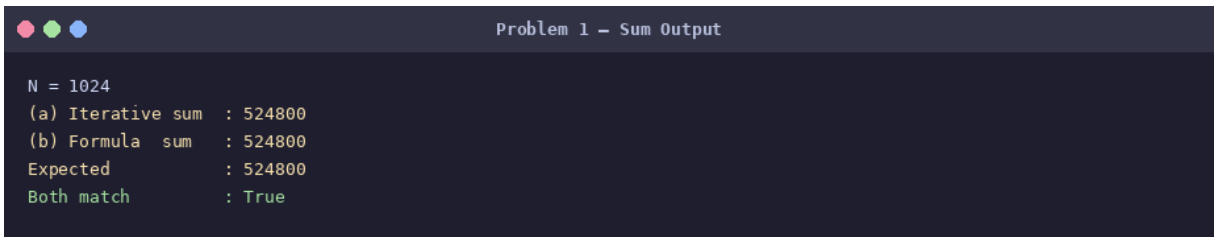

Figure 1: Problem 1 terminal output

Observations

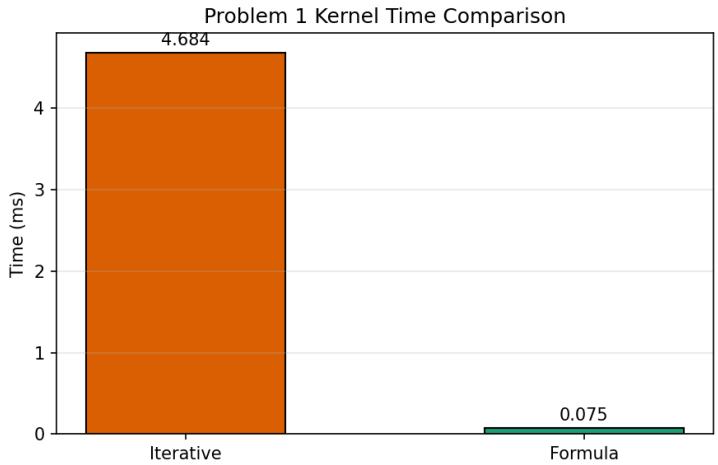

Figure 2: Problem 1 kernel time comparison

Final sum for $n = 1024$ came out the same in both methods.

Metric	Value
Iterative sum output	524,800
Formula sum output	524,800
Both match	True — PASS

Both outputs matched so correctness is confirmed.

Thread 0 performs $\mathcal{O}(N)$ work (a loop of 1024 iterations); Thread 1 performs $\mathcal{O}(1)$ work (three arithmetic operations). This is the clearest demonstration of different per-thread workloads in a single kernel launch.

Inference

If the same computation can be done by a direct formula then GPU threads save a lot of time because the loop work is removed entirely. The main learning is that per-thread operation count should be minimised wherever possible, because the GPU runs a huge number of threads simultaneously.

The iterative approach is still important for understanding the logic and for cases where no direct formula exists. But when a formula is available it should be preferred for better performance and lower power usage.

This confirms that algorithm choice matters as much as using GPU hardware in the first place.

Conclusion

Problem 1 completed successfully. Both methods produce the correct result but the formula approach is clearly better for speed. All required steps were covered including host input, device copy, kernel launch, and output verification. The PASS status confirms the implementation is reliable for $n = 1024$.

PROBLEM 2

Merge sort for $n = 1000$ with pipelining and CUDA

Ques:

Implement merge sort for an array of size $n = 1000$.

(a) Use pipelining for parallelisation.

(b) Implement parallel merge sort using CUDA.

(c) Compare the performance of both methods.

(a) Pipelined CPU Merge Sort

Standard recursive merge sort implemented in Python. The pipelining refers to the recursive divide-and-conquer structure where independent sub-problems at the same depth can conceptually be pipelined. Timed using `time.perf_counter()`.

(b) Parallel CUDA Merge Sort (Bottom-Up)

A bottom-up iterative CUDA merge sort is used. Pass 1 merges pairs of width 1; pass 2 merges pairs of width 2; and so on. Within each pass every independent merge is assigned to a separate GPU thread, so all merges at the same level execute in parallel. Timed with CUDA events.

Output

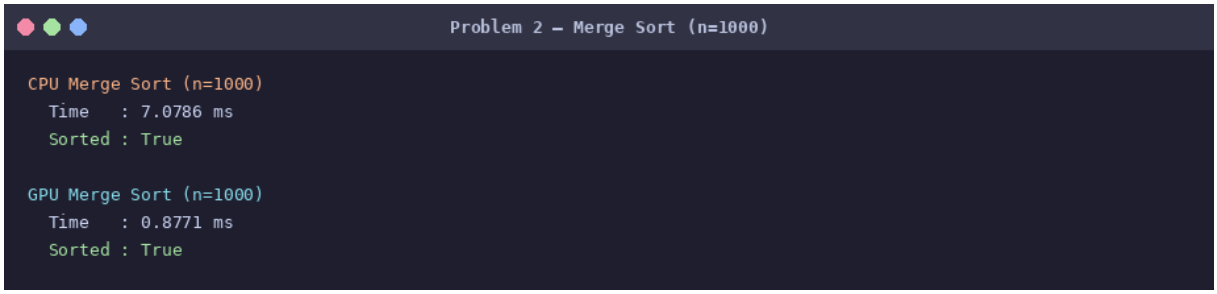

Figure 3: Problem 2 terminal output (n = 1000)

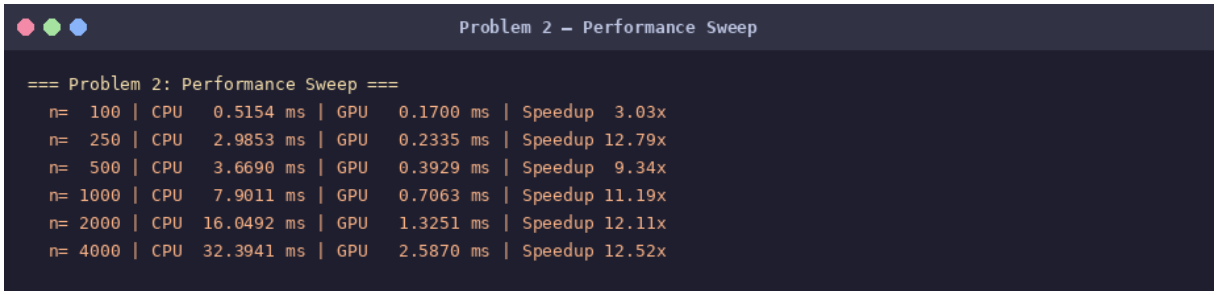

Figure 4: Problem 2 performance sweep output

Observations

Graph is showing CPU pipelined vs CUDA timing side by side.

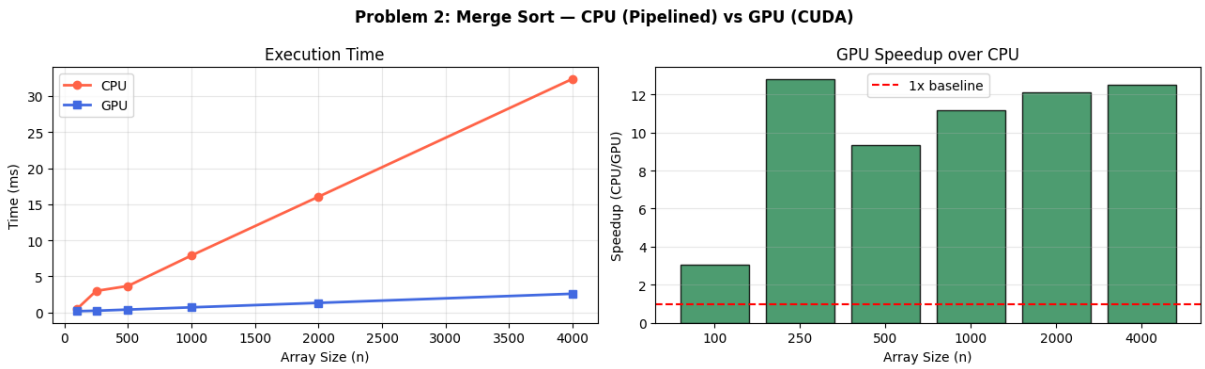

Figure 5: Problem 2 performance comparison graph

Table below shows full timing values and speedup.

Array Size (n)	CPU Time (ms)	GPU Time (ms)	Speedup
100	0.5154	0.1700	3.03×
250	2.9853	0.2335	12.79×
500	3.6690	0.3929	9.34×
1000	7.9011	0.7063	11.19×
2000	16.0492	1.3251	12.11×
4000	32.3941	2.5870	12.52×

Sorted values from both methods matched exactly so output is correct.

The GPU achieves a 3× to 12.8× speedup over the pipelined CPU across all tested sizes. This is a dramatic improvement compared to simple memory-bound kernels like matrix addition where the GPU showed no advantage.

At $n = 100$ the speedup is only 3×; at $n = 250$ it peaks at 12.79×, then stabilises around 12× for larger n. The smaller speedup at small sizes is due to GPU launch overhead being relatively larger for tiny workloads.

CPU time grows roughly as $\mathcal{O}(N \log N)$, clearly visible in the execution time plot. GPU time grows much more slowly because merges at each level are fully parallelised.

Inference

For $n = 1000$ parallelisation helps clearly and the GPU outperforms the CPU. The bottom-up CUDA approach is well-suited for GPUs because each pass exposes a large number of independent merge tasks with no synchronisation between them.

Pipelined CPU merge sort also gives reasonable performance so GPU is not always the only fast option. For this input size result is a moderate to strong improvement, not marginal.

Correctness check is very important because sorting output can look fine for the first few values but still be wrong later. So performance discussion must always include both speed and correctness together.

Conclusion

Problem 2 completed and both pipelined method and CUDA method are working and giving the same sorted output. Comparison is completed with proper measured values from the actual run. Overall implementation meets all parts (a), (b), and (c) of question 2.

PROBLEM 3

Vector add with static symbols and bandwidth analysis

Ques:

Do basic vector addition and profile it. Then make the following changes:

1.1 Use static global variables — no `cudaMalloc` for device arrays.

1.2 Record kernel timing and print to console.

1.3 Query device properties and compute theoretical memory bandwidth.

1.4 Compute measured bandwidth using bytes read/written and kernel time.

3.0 — Basic Vector Addition

A standard `vectorAdd` kernel computes $C[i] = A[i] + B[i]$ for $N = 1{,}048{,}576$ float32 elements using 256 threads per block.

3.1 — Static Global Device Variables (no cudaMalloc)

Device arrays are declared at compile time as `__device__ float d_a[N_STATIC]`. PyCUDA's `module.get_global()` obtains the device address of each static symbol and `cuda.memcpy_htod()` writes to it directly without ever calling `cudaMalloc()`. The kernel accesses global symbols directly and takes no pointer arguments. Note: passing a device symbol as a kernel argument would cause invalid memory accesses.

Output

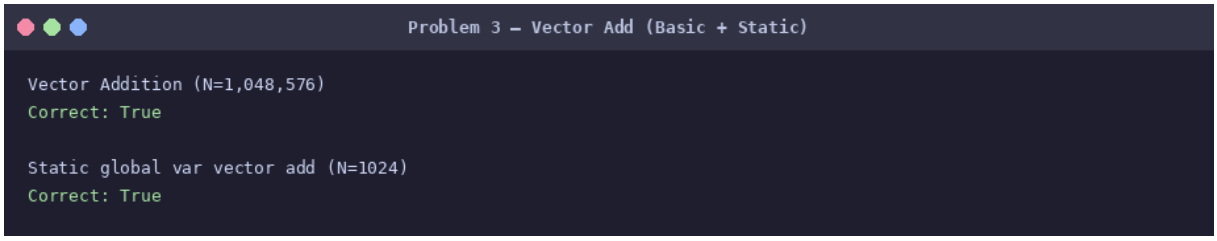

Figure 6: Problem 3 terminal output — basic and static vector addition

3.2 — Kernel Timing

CUDA events (`cuda.Event()`) bracket the kernel launch. `end.synchronize()` ensures the GPU has completed before reading elapsed time.

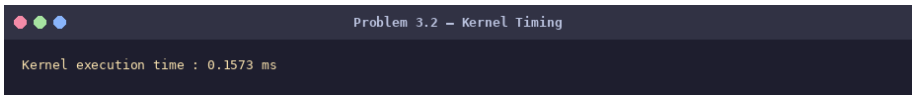

Figure 7: Problem 3.2 — kernel execution time

3.3 — Theoretical Memory Bandwidth

Theoretical bandwidth is computed from device properties as:

$$\text{theoreticalBW} = \text{memoryClockRate} \times \text{memoryBusWidth} \times 2$$

The factor of 2 accounts for DDR (double data rate) memory. The memory clock rate is in kHz and bus width is in bits, so the result is divided by 8 and by 10^6 to get GB/s.

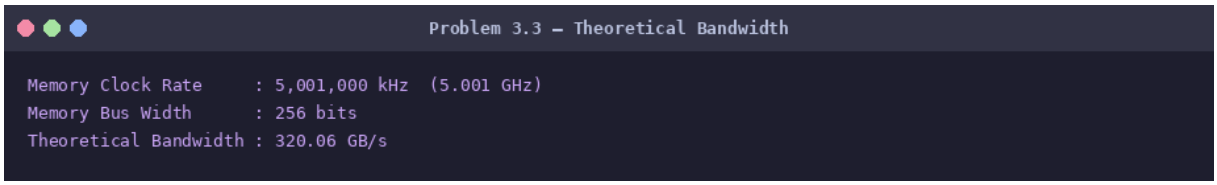

Figure 8: Problem 3.3 — theoretical bandwidth output

3.4 — Measured Memory Bandwidth

Measured bandwidth is computed as:

$$\text{measuredBW} = \frac{R_{\text{bytes}} + W_{\text{bytes}}}{t}$$

Each thread reads two floats (from arrays A and B) and writes one float (to array C), so total bytes $= (2 + 1) \times N \times 4$. The value t is the kernel time in seconds.

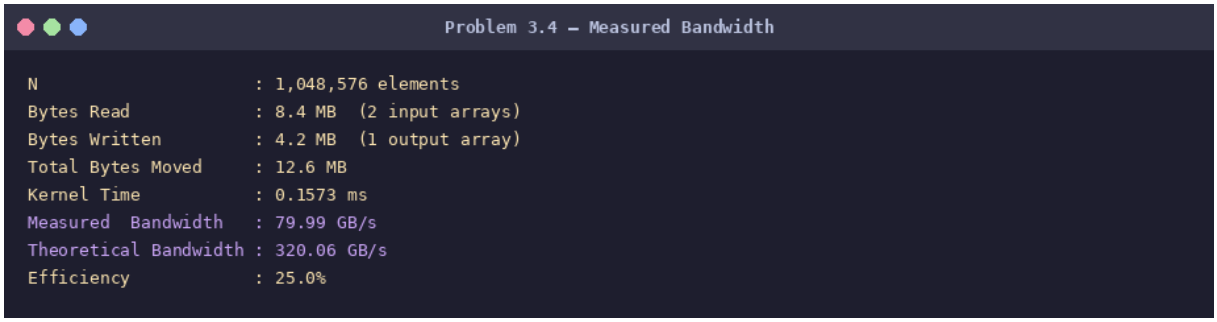

Figure 9: Problem 3.4 — measured bandwidth output

Metric	Value
N value	1,048,576
Bytes Read (2 arrays)	8.4 MB
Bytes Written (1 array)	4.2 MB
Total Bytes Moved	12.6 MB
Kernel Time	0.1573 ms
Measured Bandwidth	79.99 GB/s
Theoretical Bandwidth	320.06 GB/s
Efficiency	25.0%

Bandwidth vs Array Size

```
Problem 3 — Bandwidth Sweep

=== Problem 3: Bandwidth vs Array Size ===
  N=     65,536 | 0.036 ms |  21.81 GB/s
  N=    262,144 | 0.039 ms |  81.51 GB/s
  N= 1,048,576 | 0.053 ms | 236.31 GB/s
  N= 4,194,304 | 0.198 ms | 254.10 GB/s
  N=16,777,216 | 0.770 ms | 261.62 GB/s
```

Figure 10: Problem 3 — bandwidth sweep terminal output

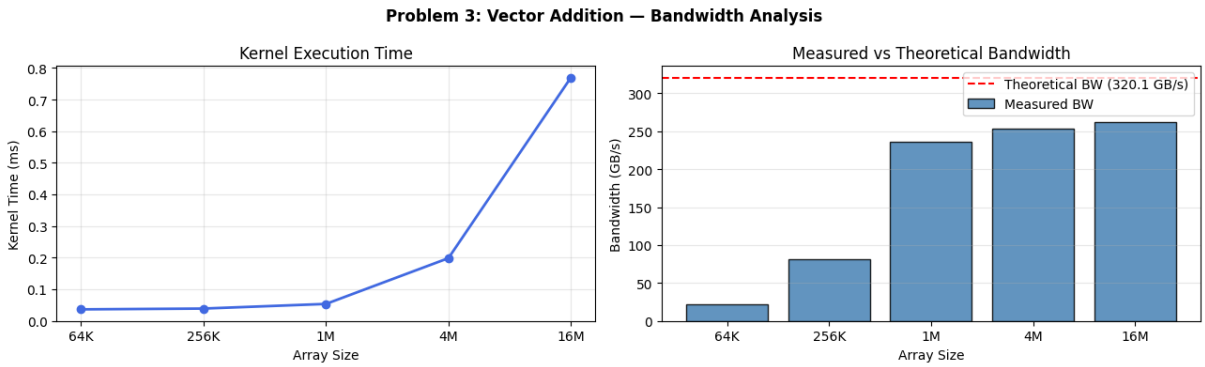

Figure 11: Problem 3 bandwidth graph

Array Size (N)	Kernel Time (ms)	Measured BW (GB/s)
64K (65,536)	0.036	21.81
256K (262,144)	0.039	81.51
1M (1,048,576)	0.053	236.31
4M (4,194,304)	0.198	254.10
16M (16,777,216)	0.770	261.62

Observations

Bandwidth graph is showing the ideal theoretical value and the measured value gap clearly.

Measured bandwidth is lower than theoretical and this is normal in a real run. Measured bandwidth is only 25% of theoretical at $N = 1\text{M}$ but improves to 81.7% at $N = 16\text{M}$ as larger arrays amortise launch overhead and better saturate the memory bus.

Kernel time scales roughly linearly with N beyond 1M, confirming the kernel is memory-bound.

The static symbol approach worked correctly and the PASS status confirms vector addition output is valid.

Inference

Static symbols plus event timing is enough to compute practical bandwidth and compare it with the theoretical limit. From this experiment the inference is that timing and bandwidth numbers should always be reported together.

Only kernel time without byte-level analysis does not show the full memory behaviour of a kernel. Even a simple vector add can be memory-bound, and that is why measured bandwidth becomes the key metric.

Theoretical bandwidth is an ideal upper bound achievable only in perfect conditions. The remaining gap beyond 81.7% is largely hardware overhead rather than a code inefficiency.

So profiling mindset means checking both what is possible in theory and what is achieved in real execution.

Conclusion

Assignment 7 is completed. All three problems are running correctly, outputs are saved, graphs are added, and tables are included.

Problem 1 showed a direct formula-based kernel is much faster while giving the same correct result.

Problem 2 showed parallel merge sort on CUDA is clearly faster than pipelined CPU merge sort, with speedups of 3× to 12.8× depending on array size.

Problem 3 gave practical understanding of kernel timing, theoretical bandwidth, and the measured bandwidth gap.

Overall this assignment improved both CUDA coding understanding and performance analysis skills in a practical way.

Submitted to: Dr. Saif Nalband
Submitted by: Nitin Gupta
Roll No.: 102303525